

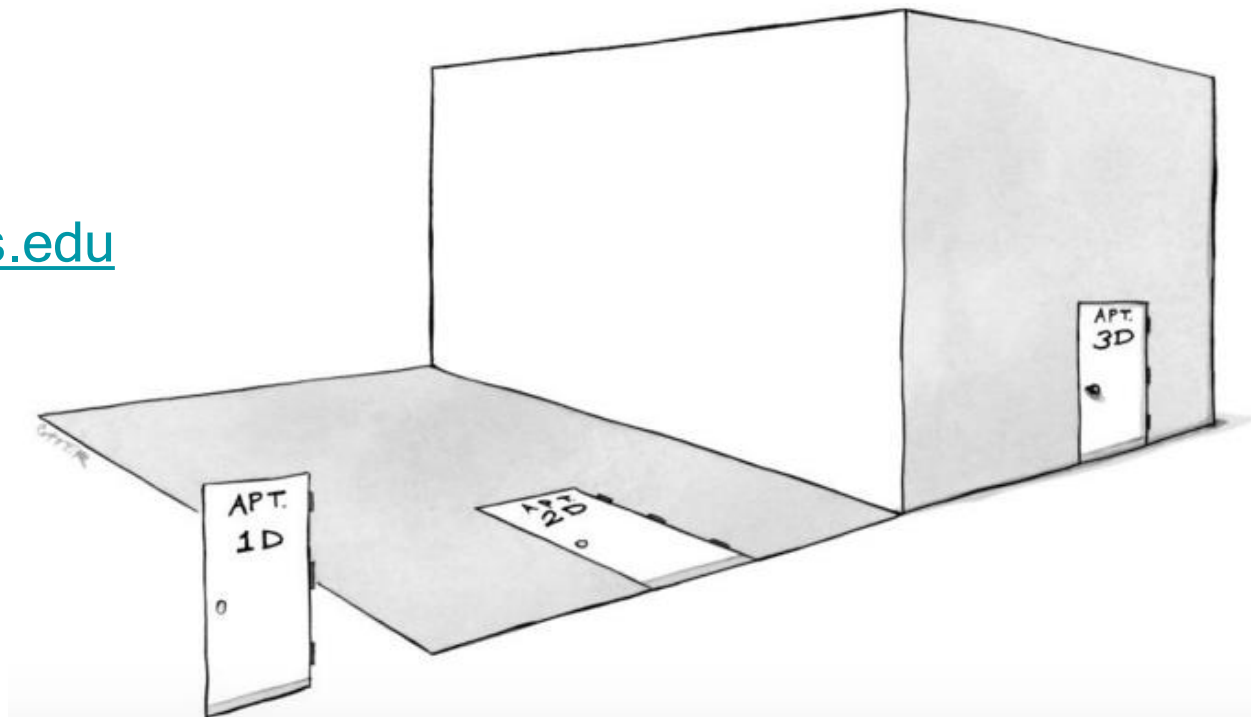
CS 532: 3D Computer Vision

Lecture 13

Enrique Dunn

edunn@stevens.edu

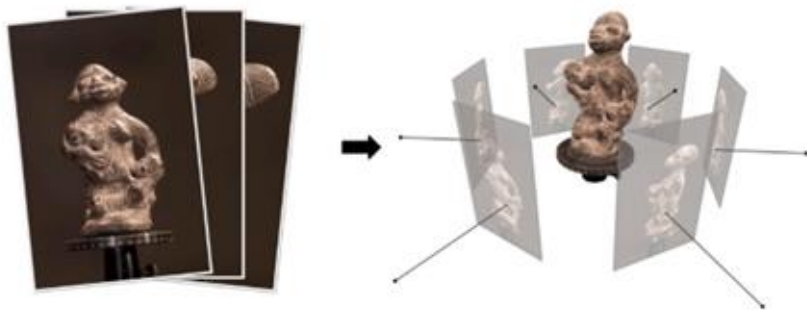
GWS 323



Outline

- Introduction to 3D Scene Reconstruction**
- Introduction to Novel view synthesis (NeRF)**
- 3D Gaussian Splatting**

Introduction to 3D Scene Reconstruction



Goal: Given a set of images of an object/scene, estimate most likely 3D shape that explains those images under assumptions of known materials, viewpoints and lighting conditions – (Furukawa & Hernandez 2015).

Multi-View Stereo: General term given to a group of techniques that use stereo correspondence as main cue and use more than 2 images.

Need for Novel View Synthesis

MVS methods obtain great geometric accuracy, but suffer with photorealism:



Larger resolution Images

Shan, Qi, et al. "The Visual Turing Test for Scene Reconstruction" *2013 International Conference on 3D Vision-3DV 2013*. IEEE, 2013

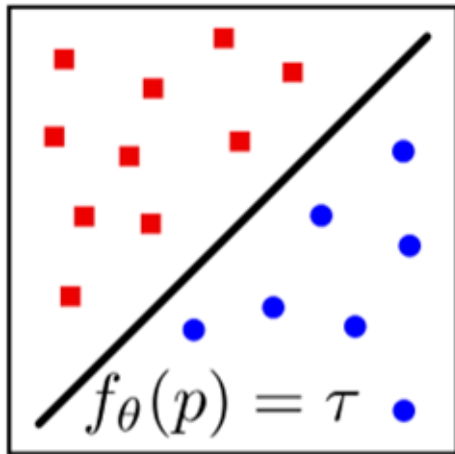
Need for Novel View Synthesis

MVS methods obtain great geometric accuracy, but suffer with photorealism:



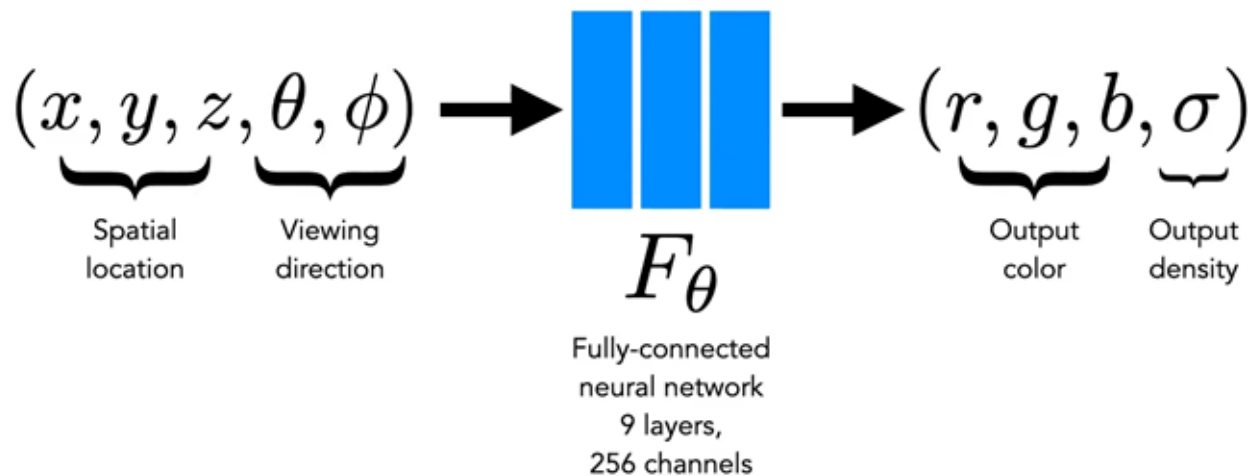
Smaller resolution Images

Neural Networks as continuous shape representations



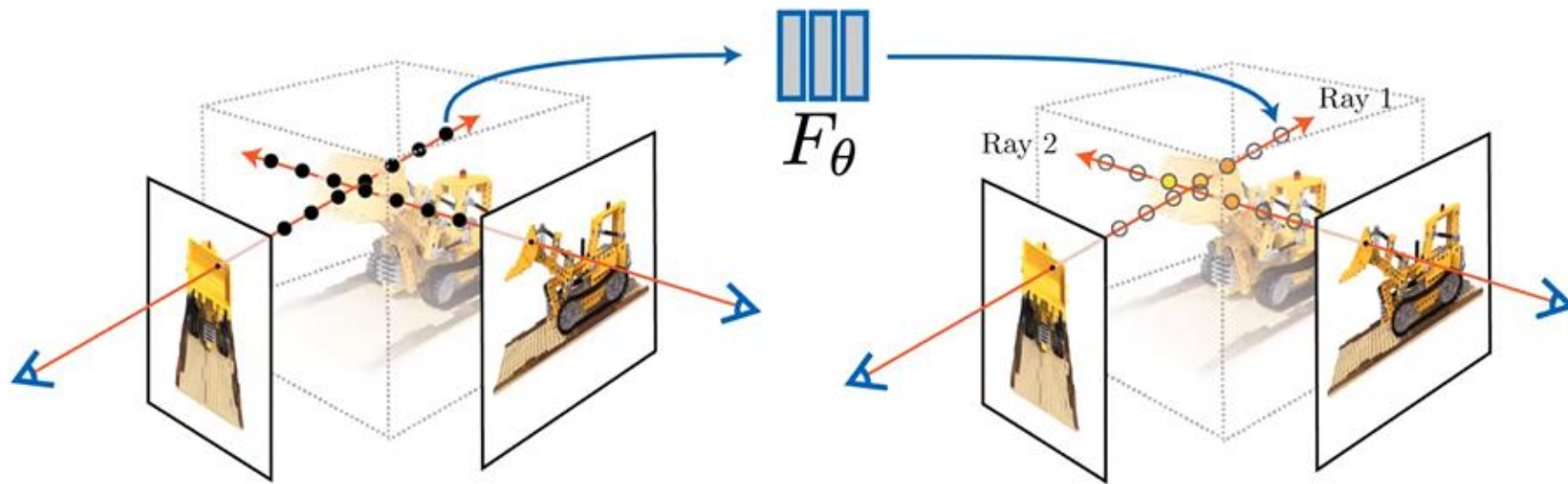
Mescheder, Lars, et al. "Occupancy networks: Learning 3d reconstruction in function space." Proceedings of the IEEE/CVF conference on computer vision and pattern recognition. 2019.

Neural Radiance Fields



Mildenhall, Ben, et al. "NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis" *Communications of the ACM* 65.1 (2021): 99-106.

Neural Radiance Fields



Mildenhall, Ben, et al. "NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis" *Communications of the ACM* 65.1 (2021): 99-106.

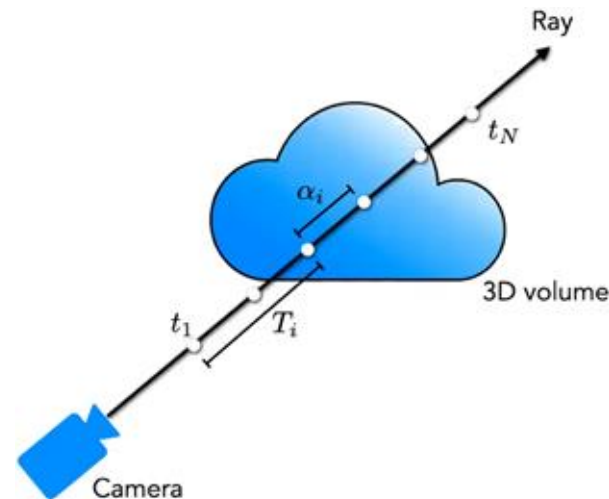
Neural Radiance Fields

Volume Rendering

Rendering model for ray $r(t) = o + td$:

$$C \approx \sum_{i=1}^N T_i \alpha_i c_i$$

weights colors



Mildenhall, Ben, et al. "NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis" *Communications of the ACM* 65.1 (2021): 99-106.

Neural Radiance Fields

Volume Rendering

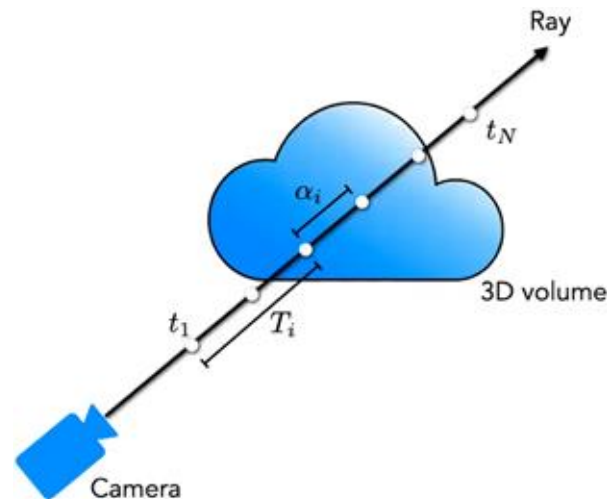
Rendering model for ray $r(t) = o + td$:

$$C \approx \sum_{i=1}^N T_i \alpha_i c_i$$

weights colors

How much light is blocked earlier along ray:

$$T_i = \prod_{j=1}^{i-1} (1 - \alpha_j)$$



Neural Radiance Fields

Volume Rendering

Rendering model for ray $r(t) = o + td$:

$$C \approx \sum_{i=1}^N T_i \alpha_i c_i$$

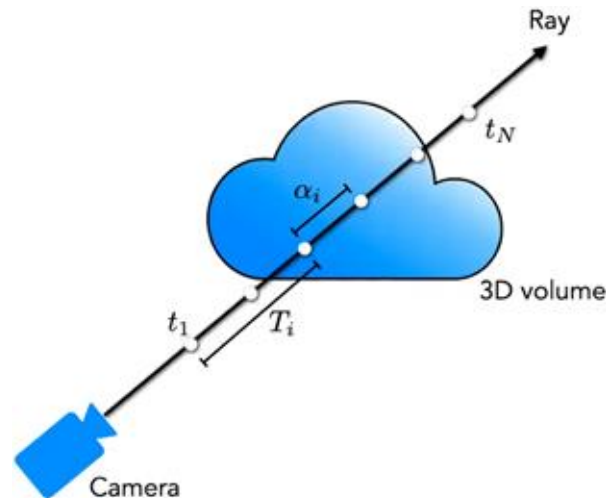
weights colors

How much light is blocked earlier along ray:

$$T_i = \prod_{j=1}^{i-1} (1 - \alpha_j)$$

How much light is contributed by ray segment i :

$$\alpha_i = 1 - e^{-\sigma_i \delta t_i}$$



Neural Radiance Fields

Volume Rendering is Differentiable

Rendering model for ray $r(t) = o + td$:

$$C \approx \sum_{i=1}^N T_i \alpha_i c_i$$

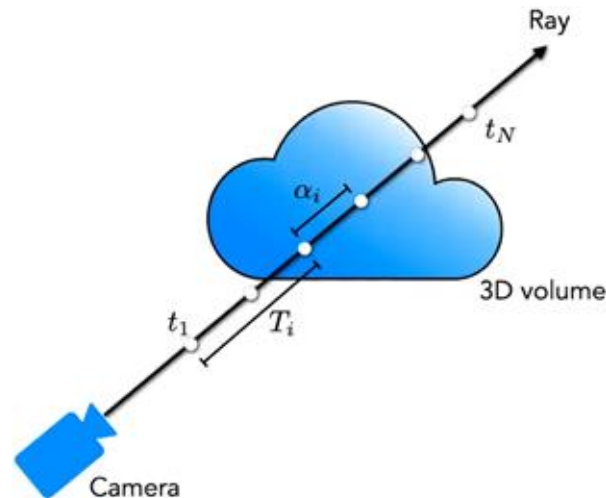
weights colors

How much light is blocked earlier along ray:

$$T_i = \prod_{j=1}^{i-1} (1 - \alpha_j)$$

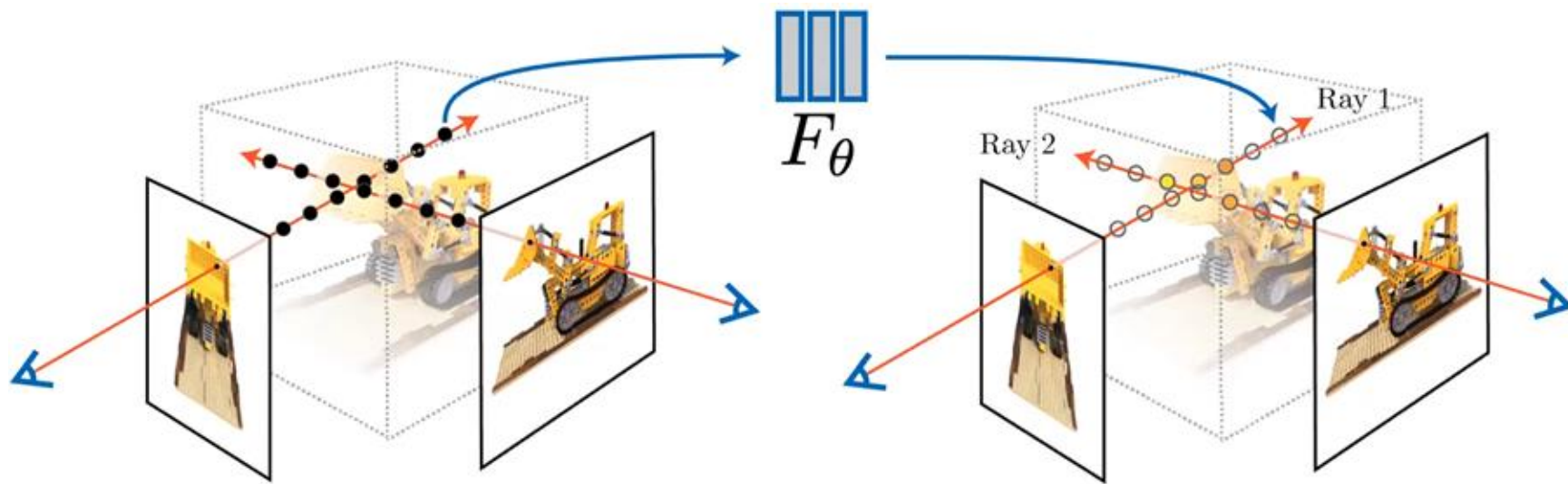
How much light is contributed by ray segment i :

$$\alpha_i = 1 - e^{-\sigma_i \delta t_i}$$



Neural Radiance Fields

Optimize rendering with gradient descent



$$\min_{\theta} \sum_i || \text{render}_i(F_{\theta}) - I_i ||^2$$

Mildenhall, Ben, et al. "NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis" *Communications of the ACM* 65.1 (2021): 99-106.

Neural Radiance Fields

State of the art Photorealism



Mildenhall, Ben, et al. "NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis" *Communications of the ACM* 65.1 (2021): 99-106.

Neural Radiance Fields

Problem: Very Slow to optimize. State of The Art NeRF methods take ~48 hours¹ per scene!

[1] Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, George Drettakis. "3D Gaussian Splatting for Real-Time Radiance Field Rendering" ACM Trans. Graph. 42.4 (2023): 139-1

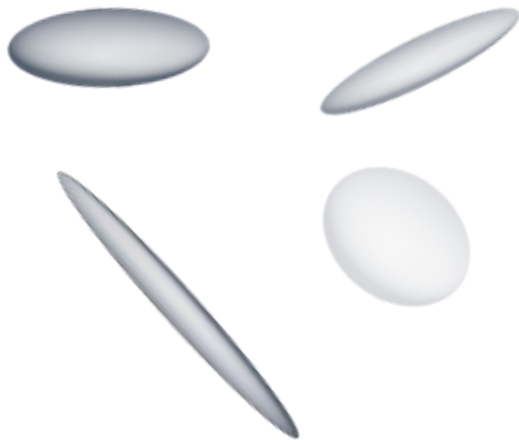
Overview

What is 3DGS ?

3D Gaussian Splatting (3DGS) [1] is both a ***novel view synthesis*** and ***fancy point-based rendering [2]*** technique used to capture real-life data into a 3D space and render it in real-time.

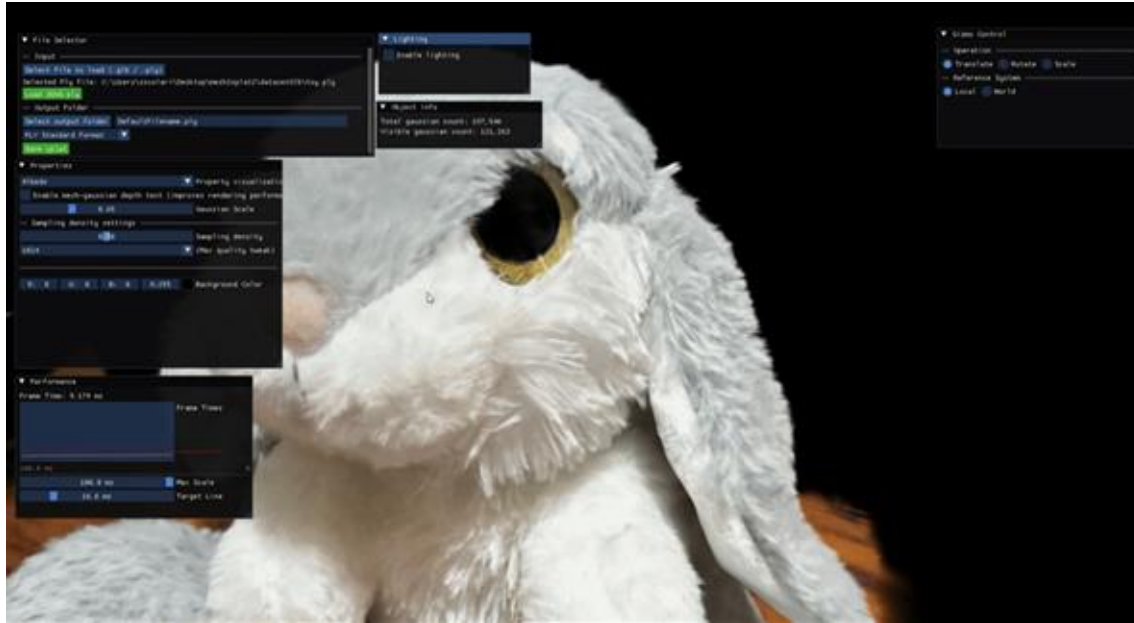
Why 3DGS?

More **generalizable primitive** than triangles



Why 3DGS?

More generalizable primitive than triangles



Why 3DGS?

More generalizable primitive than triangles

Appealing mathematical properties (filterability)

Why 3DGS?

More generalizable primitive than triangles

Appealing mathematical properties (filterability)

No connectivity is good for parallelization [5]

Why 3DGS?

More generalizable primitive than triangles

Appealing mathematical properties (filterability)

No connectivity is good for parallelization

Great for Novel-View-Synthesis (Real world $\rightarrow$ 3D space)

Point Based Rendering

Point Sample Rendering: Efficient for rendering disconnected and unstructured geometry like point clouds. But it faces challenges like **holes**, **aliasing**, and **discontinuities**.



Image source [25]

Point Based Rendering

Splatting Techniques: Address rendering issues by expanding point primitives into larger shapes (e.g., discs, ellipsoids) to improve quality.



Image source [23]

Inverse Rendering

The task of **recovering** the properties of a scene, such as *shape*, *material*, and *lighting*, **from an image or a video**.

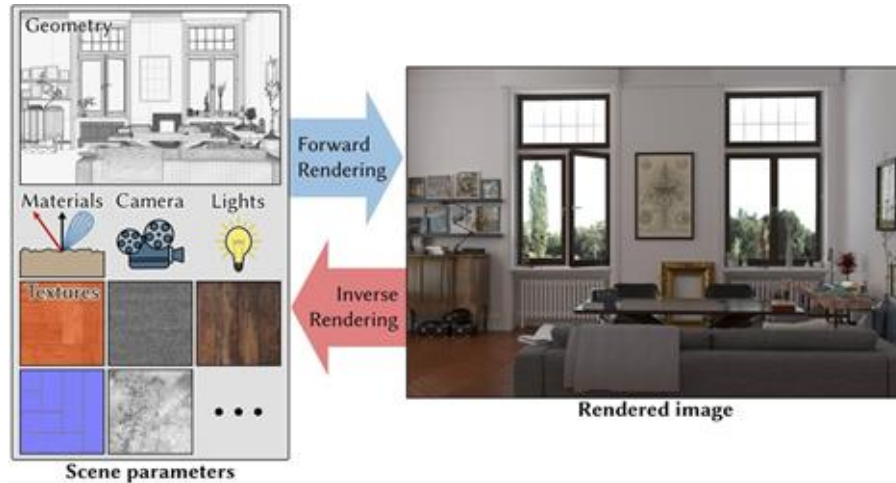


Image source [24]

Inverse Rendering

Recent inverse rendering methods focus on **computing gradients w.r.t a set of parameters of 3D objects** for optimization

Essential in inverse rendering in order to refine the scene reconstruction from images, is a differentiable renderer.

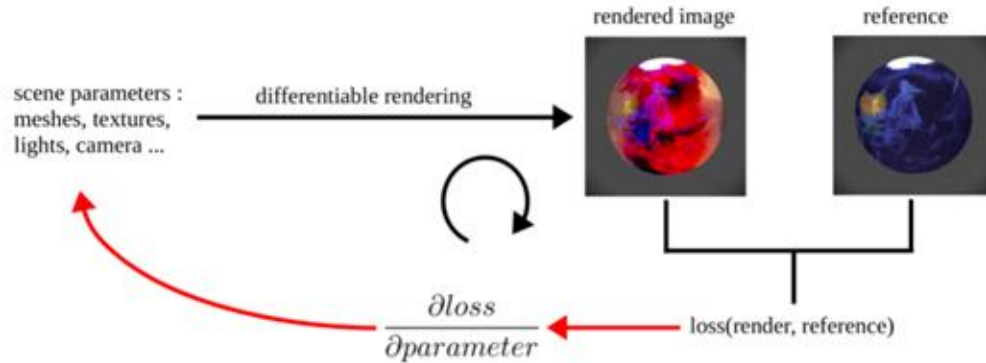


Image source [22]

Inverse Rendering

Differentiable Rendering: The goal of differentiable rendering is to provide a **differentiable rendering function** → compute its derivative with regard to different scene parameters.

$$I(\theta) = \mathcal{R}(p(\theta))$$

Inverse Rendering

Multi-View Methods: Leveraging **volumetric rendering** (e.g., **NeRFs**, **Signed Distance Functions**) improves reconstruction but faces challenges with **spatially-varying reflectance** accuracy and **slow training** due to extensive sampling [17].

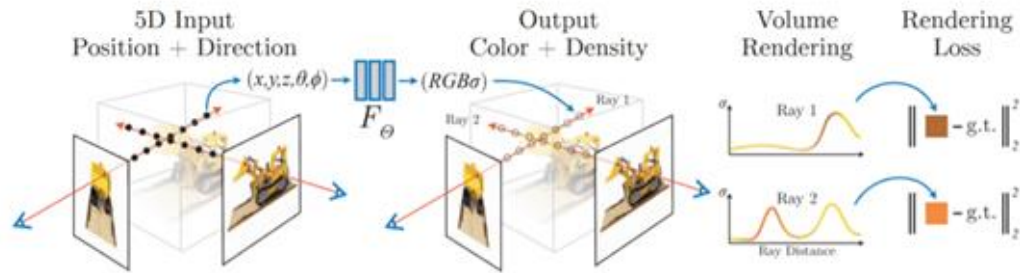


Image source [21]

Inverse Rendering

3DGS: excellent explicit representation. They are **fully differentiable** when unbounded. When constrained, can be rasterized efficiently using 2D projections and standard alpha-blending. Combining **flexibility** and **computational efficiency**.

Novel View Synthesis

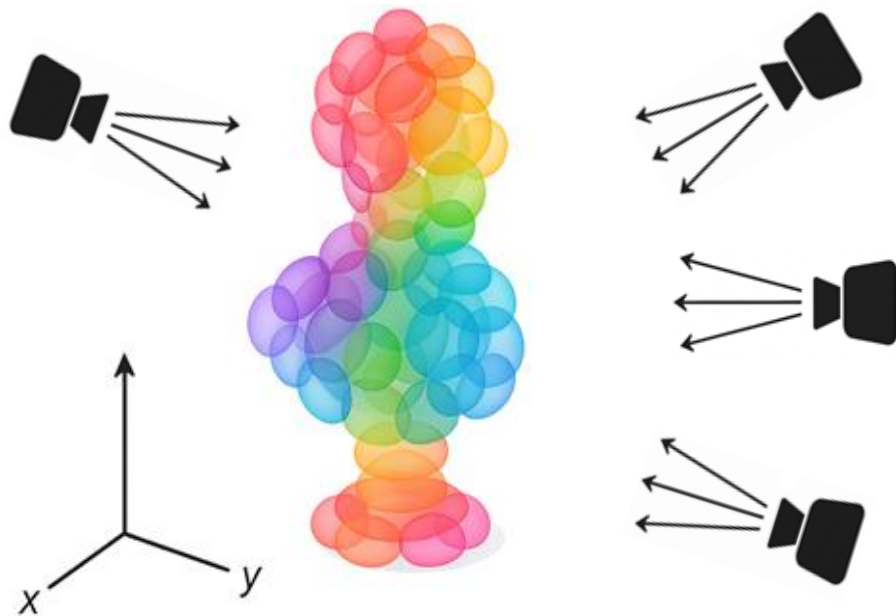
NERFs: implicit scene representation

3DGSs: explicit scene representation → rasterization!

3D Gaussian Splatting

Scene Representation

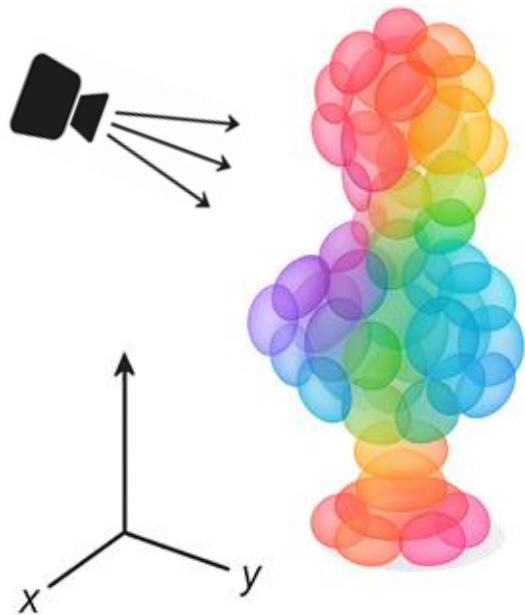
Collection of 3D primitives:



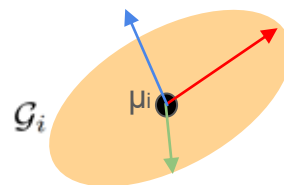
3D Gaussian Splatting

Scene Representation

Collection of 3D primitives:



Each primitive is a 3D Gaussian:



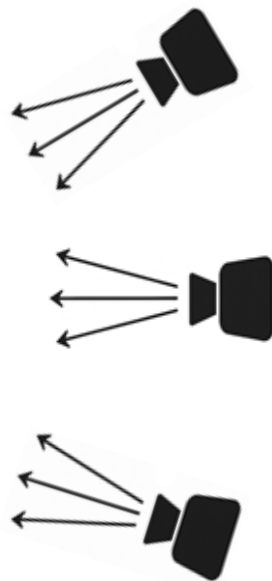
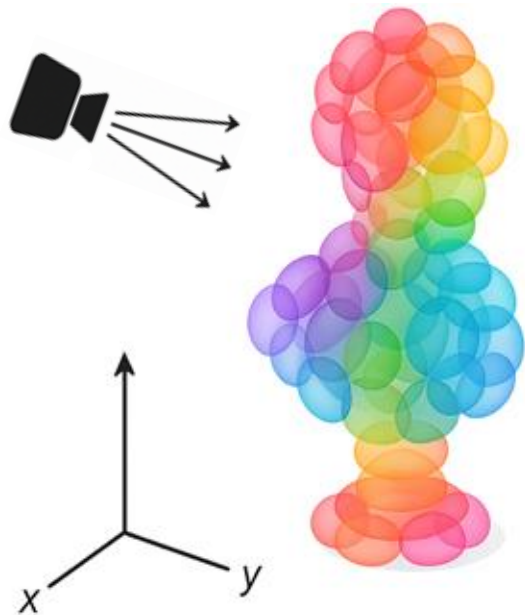
Defined at x as:

$$\mathcal{G}_i(x|\mu_i, \Sigma_i) = e^{-\frac{1}{2}(x-\mu_i)^\top \Sigma_i^{-1}(x-\mu_i)}$$

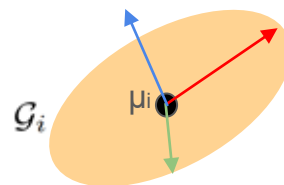
3D Gaussian Splatting

Scene Representation

Collection of 3D primitives:



Each primitive is a 3D Gaussian:



Defined at x as:

$$\mathcal{G}_i(x|\mu_i, \Sigma_i) = e^{-\frac{1}{2}(x-\mu_i)^\top \Sigma_i^{-1}(x-\mu_i)}$$

With 3x3 Covariance Σ_i parameterized as:

$$\Sigma_i = R_i S_i S_i^\top R_i^\top$$

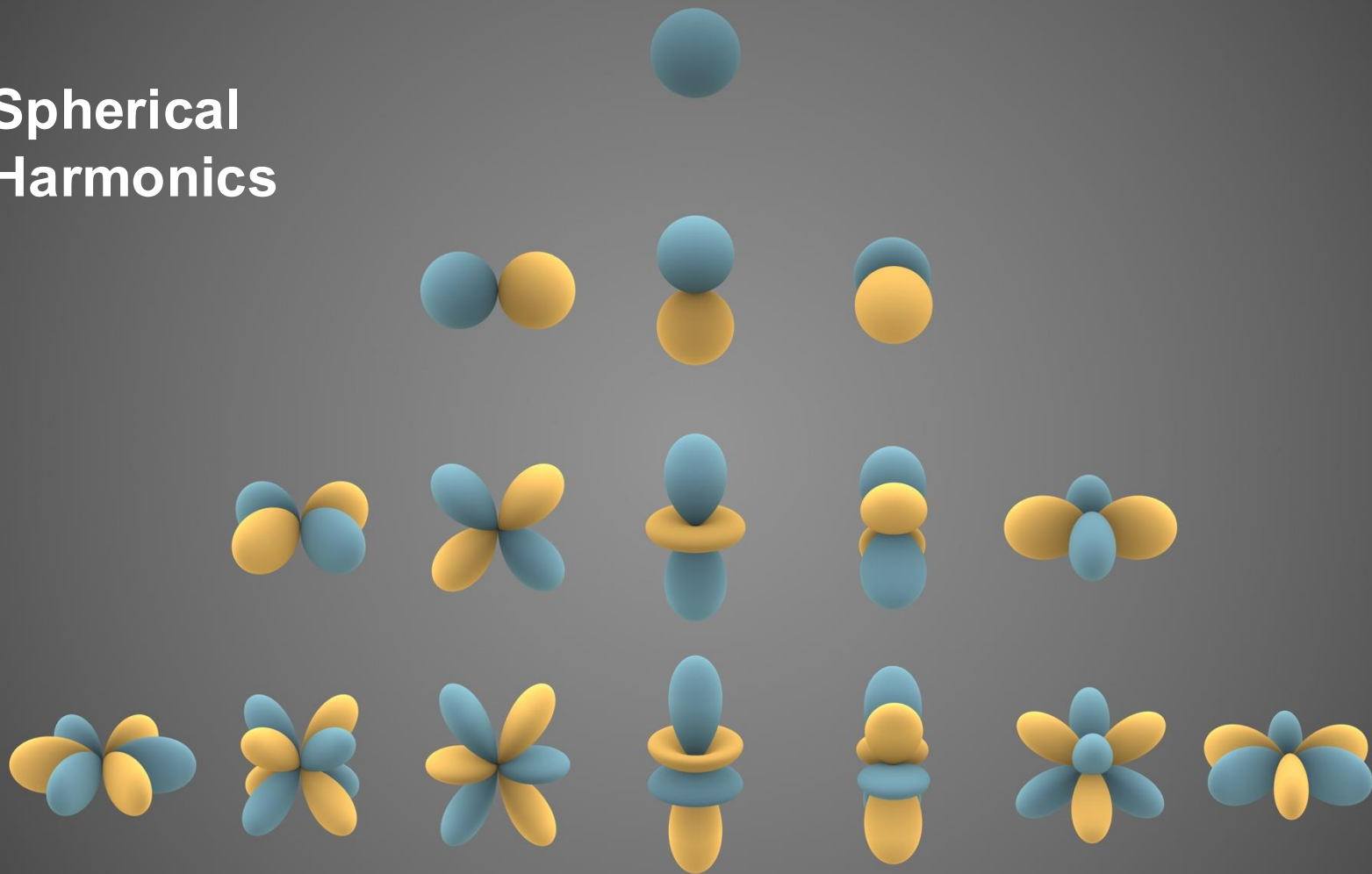
3D Gaussian Splatting

Optimizable parameters –

Each 3D Gaussian has 58 optimizable parameters:

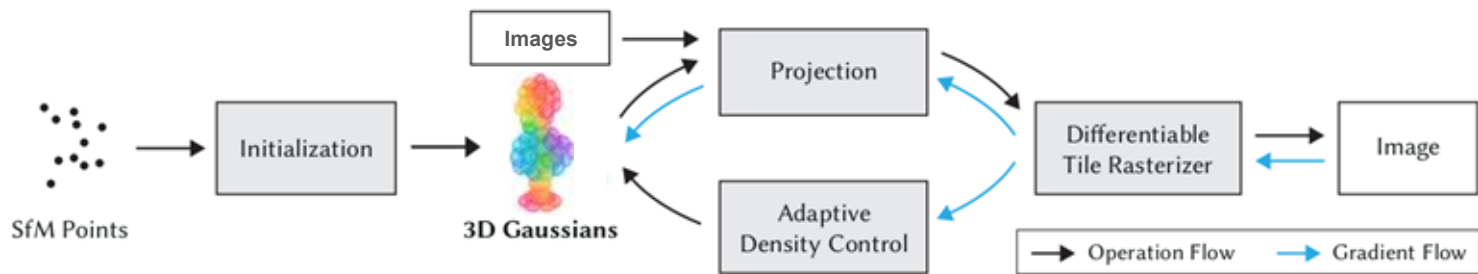
- Mean μ_i – 3 parameters x, y, z
- Scale S_i – 3 parameters s_x, s_y, s_z
- Rotation R_i – 3 parameters represented as a quaternion: q_w, q_x, q_y, q_z
- View dependent Spherical Harmonics representing color – 48 parameters
- Opacity O_i – 1 parameter.

Spherical Harmonics



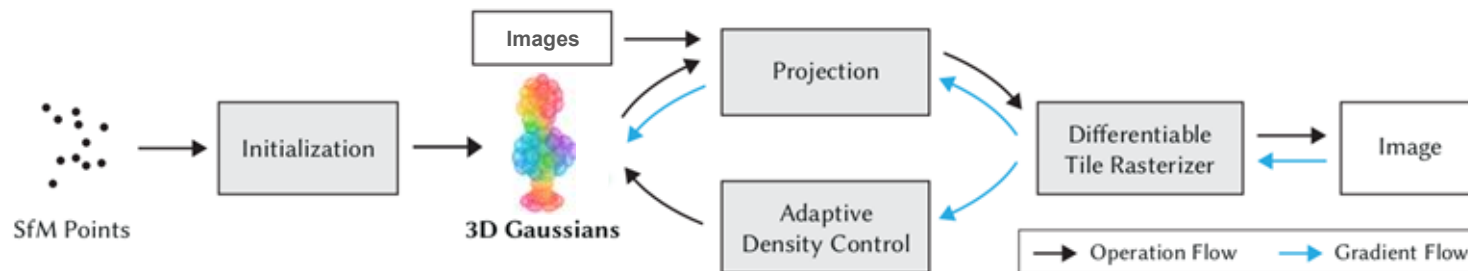
3D Gaussian Splatting

Overview



3D Gaussian Splatting

Overview

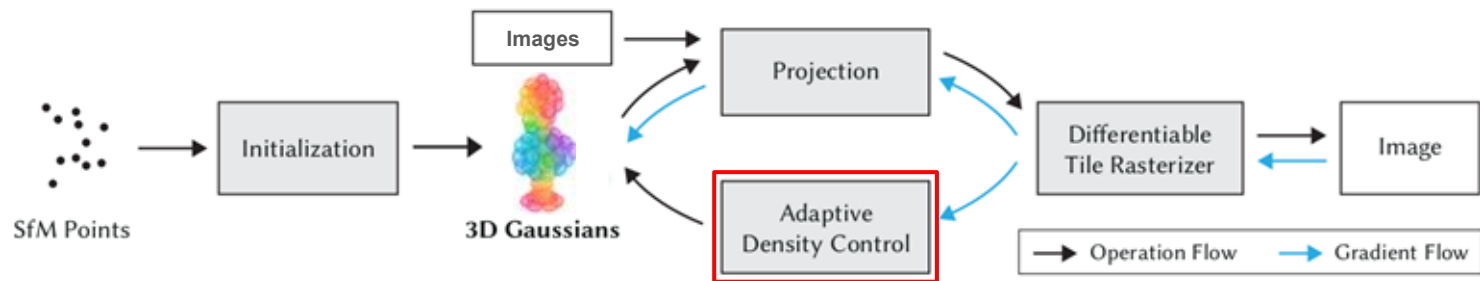


- **Input:** Set of static images of the scene, calibrated cameras and sparse point cloud. The latter two are obtained from standard off the shelf Structure from Motion (SfM) softwares.

- **Initialization:** Define 3D Gaussians as isotropic spheres, with scales initialized by its 3 nearest neighbors, and rotation matrix set to identity (meaning unoriented).

3D Gaussian Splatting

Overview



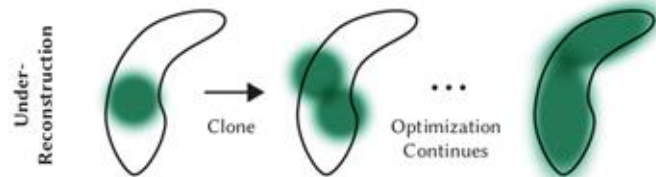
3D Gaussian Splatting

Adaptive Density Control

- Need to densify initial set of sparse point clouds for good rendering quality, but also need to represent the scene compactly.

- 2 types of regions to look out for:

- under-reconstructed regions (represented by small & sparse Gaussians).
- over-reconstructed regions (represented by few large Gaussians).



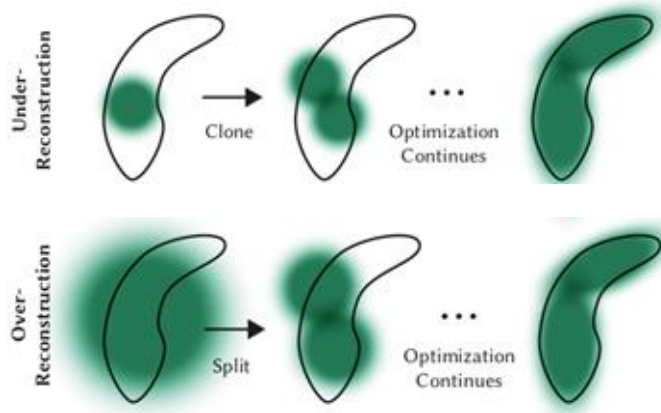
3D Gaussian Splatting

Adaptive Density Control

- Need to densify initial set of sparse point clouds for good rendering quality, but also need to represent the scene compactly.

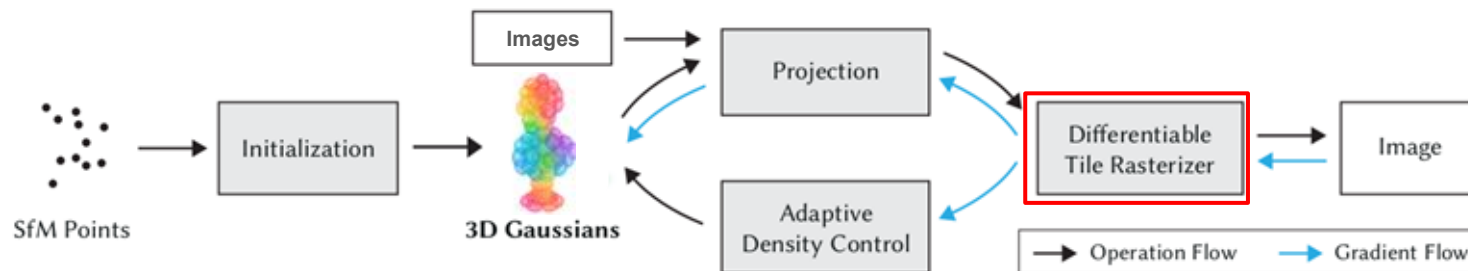
- 2 types of regions to look out for:

- under-reconstructed regions (represented by small & sparse Gaussians).
- over-reconstructed regions (represented by few large Gaussians).



3D Gaussian Splatting

Overview



- **Input:** Set of static images of the scene, calibrated cameras and sparse point cloud. The latter 2 is obtained from standard off the shelf Structure from Motion (SfM) softwares.

- **Initialization:** Define 3D Gaussians as isotropic spheres, with scales initialized by its 3 nearest neighbors, and rotation matrix set to identity (meaning unoriented).

3D Gaussian Splatting

Recap: Image formation same as in NeRF

Rendering model for ray $r(t) = o + td$:

$$C \approx \sum_{i=1}^N T_i \alpha_i c_i$$

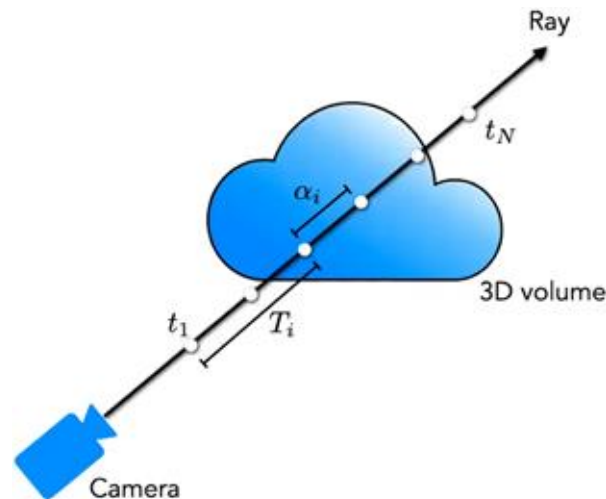
weights colors

How much light is blocked earlier along ray:

$$T_i = \prod_{j=1}^{i-1} (1 - \alpha_j)$$

How much light is contributed by ray segment i :

$$\alpha_i = 1 - e^{-\sigma_i \delta t_i}$$



3D Gaussian Splatting

Fast Differentiable Rasterizer

- Loss is between rendered image and ground truth image, therefore need to propagate gradients through the rasterizer back to the 3D Gaussians
- Rendering happens every iteration. Needs to have minimum performance overhead.

3D Gaussian Splatting

Fast Differentiable Rasterizer

Algorithm 2 GPU software rasterization of 3D Gaussians

w, h : width and height of the image to rasterize

M, S : Gaussian means and covariances in world space

C, A : Gaussian colors and opacities

V : view configuration of current camera

```
function RASTERIZE( $w, h, M, S, C, A, V$ )  
  CullGaussian( $p, V$ )                                ▶ Frustum Culling  
   $M', S' \leftarrow$  ScreenspaceGaussians( $M, S, V$ )      ▶ Transform  
   $T \leftarrow$  CreateTiles( $w, h$ )  
   $L, K \leftarrow$  DuplicateWithKeys( $M', T$ )             ▶ Indices and Keys  
  SortByKeys( $K, L$ )                                    ▶ Globally Sort  
   $R \leftarrow$  IdentifyTileRanges( $T, K$ )  
   $I \leftarrow 0$                                        ▶ Init Canvas  
  for all Tiles  $t$  in  $I$  do  
    for all Pixels  $i$  in  $t$  do  
       $r \leftarrow$  GetTileRange( $R, t$ )  
       $I[i] \leftarrow$  BlendInOrder( $i, L, r, K, M', S', C, A$ )  
    end for  
  end for  
  return  $I$   
end function
```

3D Gaussian Splatting

Fast Differentiable Rasterizer

Algorithm 2 GPU software rasterization of 3D Gaussians

w, h : width and height of the image to rasterize

M, S : Gaussian means and covariances in world space

C, A : Gaussian colors and opacities

V : view configuration of current camera

function RASTERIZE(w, h, M, S, C, A, V)

 CullGaussian(p, V) ▷ Frustum Culling

$M', S' \leftarrow$ ScreenspaceGaussians(M, S, V) ▷ Transform

$T \leftarrow$ CreateTiles(w, h)

$L, K \leftarrow$ DuplicateWithKeys(M', T) ▷ Indices and Keys

 SortByKeys(K, L) ▷ Globally Sort

$R \leftarrow$ IdentifyTileRanges(T, K)

$I \leftarrow 0$ ▷ Init Canvas

for all Tiles t **in** I **do**

for all Pixels i **in** t **do**

$r \leftarrow$ GetTileRange(R, t)

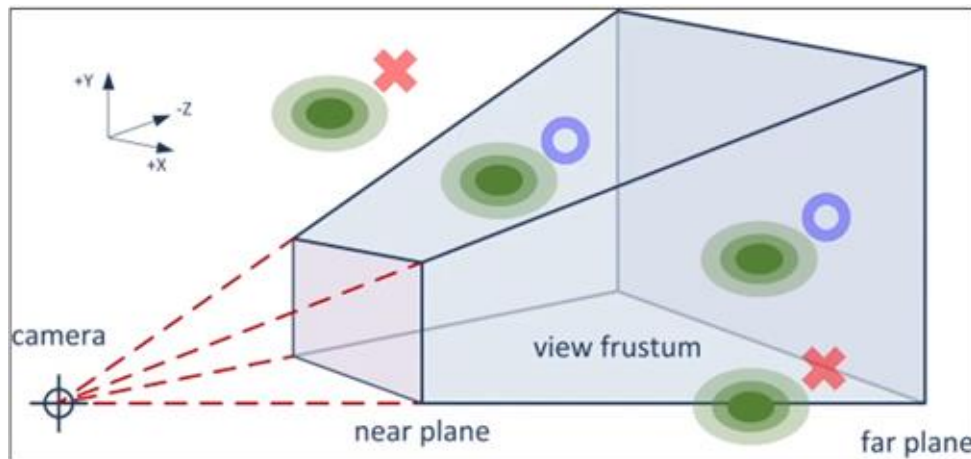
$I[i] \leftarrow$ BlendInOrder(i, L, r, K, M', S', C, A)

end for

end for

return I

end function



3D Gaussian Splatting

Fast Differentiable Rasterizer

Algorithm 2 GPU software rasterization of 3D Gaussians

w, h : width and height of the image to rasterize

M, S : Gaussian means and covariances in world space

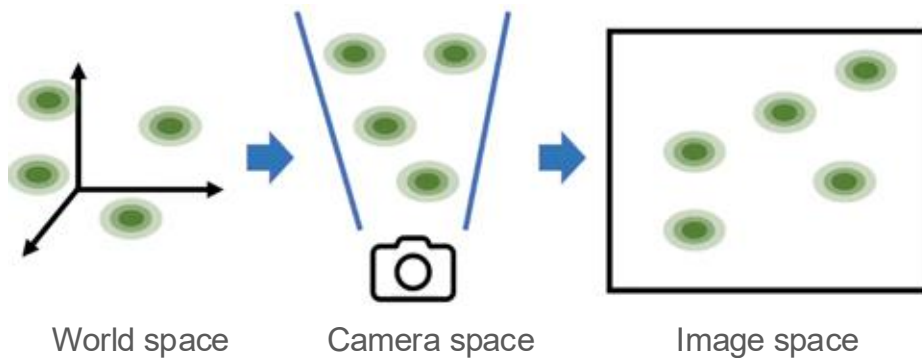
C, A : Gaussian colors and opacities

V : view configuration of current camera

```
function RASTERIZE( $w, h, M, S, C, A, V$ )  
  CullGaussian( $p, V$ )                                ▶ Frustum Culling  
   $M', S' \leftarrow$  ScreenspaceGaussians( $M, S, V$ )      ▶ Transform  
   $T \leftarrow$  CreateTiles( $w, h$ )  
   $L, K \leftarrow$  DuplicateWithKeys( $M', T$ )             ▶ Indices and Keys  
  SortByKeys( $K, L$ )                                   ▶ Globally Sort  
   $R \leftarrow$  IdentifyTileRanges( $T, K$ )  
   $I \leftarrow 0$                                        ▶ Init Canvas  
  for all Tiles  $t$  in  $I$  do  
    for all Pixels  $i$  in  $t$  do  
       $r \leftarrow$  GetTileRange( $R, t$ )  
       $I[i] \leftarrow$  BlendInOrder( $i, L, r, K, M', S', C, A$ )  
    end for  
  end for  
  return  $I$   
end function
```

Screenspace Gaussians:

- Projects the 3D Gaussians to image space (splatting)



3D Gaussian Splatting

Fast Differentiable Rasterizer

Algorithm 2 GPU software rasterization of 3D Gaussians

w, h : width and height of the image to rasterize

M, S : Gaussian means and covariances in world space

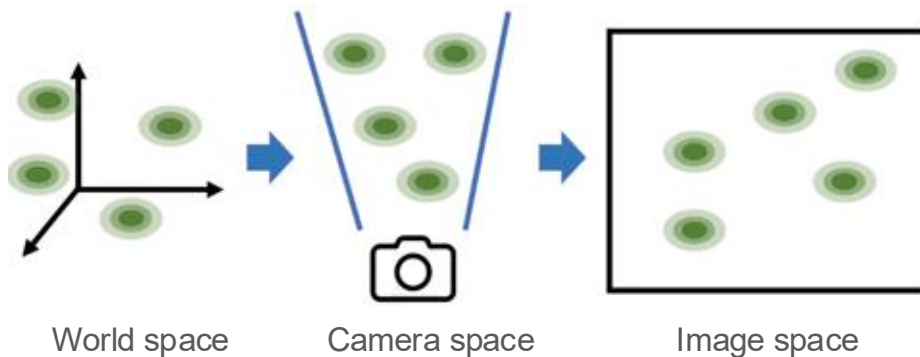
C, A : Gaussian colors and opacities

V : view configuration of current camera

```
function RASTERIZE( $w, h, M, S, C, A, V$ )  
  CullGaussian( $p, V$ )                                 $\triangleright$  Frustum Culling  
   $M', S' \leftarrow$  ScreenspaceGaussians( $M, S, V$ )       $\triangleright$  Transform  
   $T \leftarrow$  CreateTiles( $w, h$ )  
   $L, K \leftarrow$  DuplicateWithKeys( $M', T$ )             $\triangleright$  Indices and Keys  
  SortByKeys( $K, L$ )                                   $\triangleright$  Globally Sort  
   $R \leftarrow$  IdentifyTileRanges( $T, K$ )  
   $I \leftarrow 0$                                         $\triangleright$  Init Canvas  
  for all Tiles  $t$  in  $I$  do  
    for all Pixels  $i$  in  $t$  do  
       $r \leftarrow$  GetTileRange( $R, t$ )  
       $I[i] \leftarrow$  BlendInOrder( $i, L, r, K, M', S', C, A$ )  
    end for  
  end for  
  return  $I$   
end function
```

Screenspace Gaussians:

- Projects the 3D Gaussians to image space (splatting)
- 3D Gaussians do not project to 2D Gaussians under perspective projection, but do so under affine projection:



3D Gaussian Splatting

Fast Differentiable Rasterizer

Algorithm 2 GPU software rasterization of 3D Gaussians

w, h : width and height of the image to rasterize

M, S : Gaussian means and covariances in world space

C, A : Gaussian colors and opacities

V : view configuration of current camera

```
function RASTERIZE( $w, h, M, S, C, A, V$ )  
  CullGaussian( $p, V$ )                                ▶ Frustum Culling  
   $M', S' \leftarrow$  ScreenspaceGaussians( $M, S, V$ )      ▶ Transform  
   $T \leftarrow$  CreateTiles( $w, h$ )  
   $L, K \leftarrow$  DuplicateWithKeys( $M', T$ )             ▶ Indices and Keys  
  SortByKeys( $K, L$ )                                   ▶ Globally Sort  
   $R \leftarrow$  IdentifyTileRanges( $T, K$ )  
   $I \leftarrow 0$                                        ▶ Init Canvas  
  for all Tiles  $t$  in  $I$  do  
    for all Pixels  $i$  in  $t$  do  
       $r \leftarrow$  GetTileRange( $R, t$ )  
       $I[i] \leftarrow$  BlendInOrder( $i, L, r, K, M', S', C, A$ )  
    end for  
  end for  
  return  $I$   
end function
```

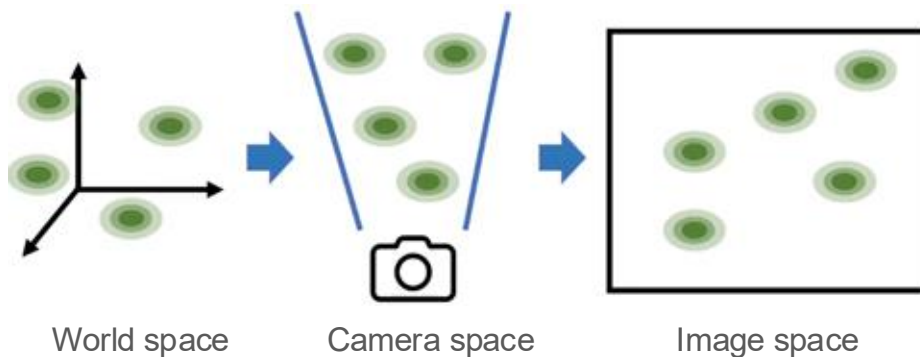
Screenspace Gaussians:

- Projects the 3D Gaussians to image space (splatting)

- 3D Gaussians do not project to 2D Gaussians under perspective projection, but do so under affine projection:

$$\Sigma' = J W \Sigma W^T J^T$$

- Σ' : Cov matrix in cam coordinates
- W : World to Cam Transform
- J : Jacobian of the Affine approximation of projective transform.



3D Gaussian Splatting

Fast Differentiable Rasterizer

Algorithm 2 GPU software rasterization of 3D Gaussians

w, h : width and height of the image to rasterize

M, S : Gaussian means and covariances in world space

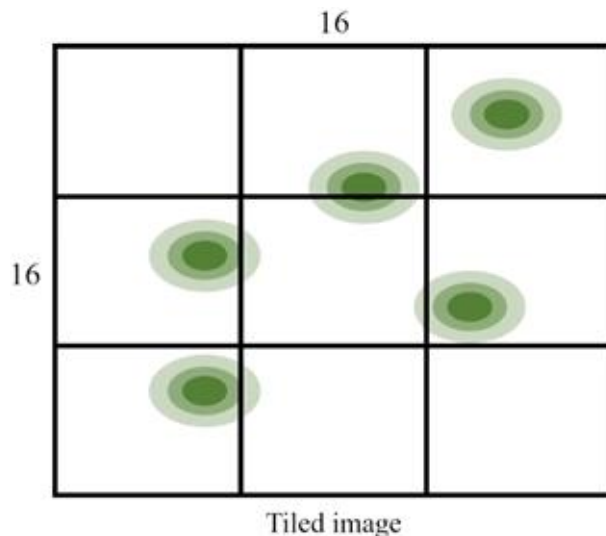
C, A : Gaussian colors and opacities

V : view configuration of current camera

```
function RASTERIZE( $w, h, M, S, C, A, V$ )  
  CullGaussian( $p, V$ )                                ▶ Frustum Culling  
   $M', S' \leftarrow$  ScreenspaceGaussians( $M, S, V$ )      ▶ Transform  
   $T \leftarrow$  CreateTiles( $w, h$ )  
   $L, K \leftarrow$  DuplicateWithKeys( $M', T$ )             ▶ Indices and Keys  
  SortByKeys( $K, L$ )                                   ▶ Globally Sort  
   $R \leftarrow$  IdentifyTileRanges( $T, K$ )  
   $I \leftarrow 0$                                        ▶ Init Canvas  
  for all Tiles  $t$  in  $I$  do  
    for all Pixels  $i$  in  $t$  do  
       $r \leftarrow$  GetTileRange( $R, t$ )  
       $I[i] \leftarrow$  BlendInOrder( $i, L, r, K, M', S', C, A$ )  
    end for  
  end for  
  return  $I$   
end function
```

Create image tiles:

- Subdivide image into 16x16 tiles for higher GPU parallelism



3D Gaussian Splatting

Fast Differentiable Rasterizer

Algorithm 2 GPU software rasterization of 3D Gaussians

w, h : width and height of the image to rasterize

M, S : Gaussian means and covariances in world space

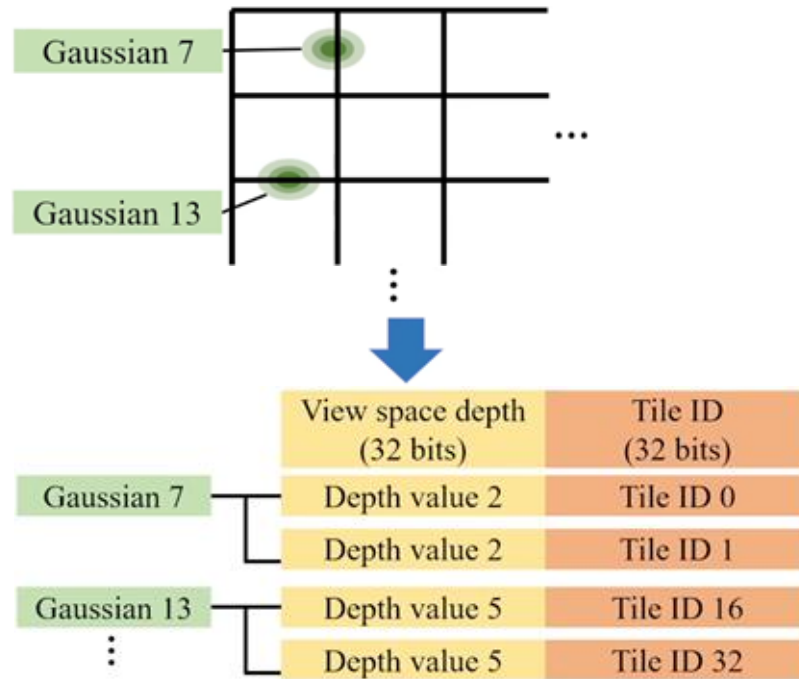
C, A : Gaussian colors and opacities

V : view configuration of current camera

```
function RASTERIZE( $w, h, M, S, C, A, V$ )  
  CullGaussian( $p, V$ )                                ▶ Frustum Culling  
   $M', S' \leftarrow$  ScreenspaceGaussians( $M, S, V$ )      ▶ Transform  
   $T \leftarrow$  CreateTiles( $w, h$ )  
   $L, K \leftarrow$  DuplicateWithKeys( $M', T$ )             ▶ Indices and Keys  
  SortByKey( $K, L$ )                                    ▶ Globally Sort  
   $R \leftarrow$  IdentifyTileRanges( $T, K$ )  
   $I \leftarrow 0$                                        ▶ Init Canvas  
  for all Tiles  $t$  in  $I$  do  
    for all Pixels  $i$  in  $t$  do  
       $r \leftarrow$  GetTileRange( $R, t$ )  
       $I[i] \leftarrow$  BlendInOrder( $i, L, r, K, M', S', C, A$ )  
    end for  
  end for  
  return  $I$   
end function
```

Duplicate with keys:

- Assign keys for each Gaussian visible in each tile



3D Gaussian Splatting

Fast Differentiable Rasterizer

Algorithm 2 GPU software rasterization of 3D Gaussians

w, h : width and height of the image to rasterize

M, S : Gaussian means and covariances in world space

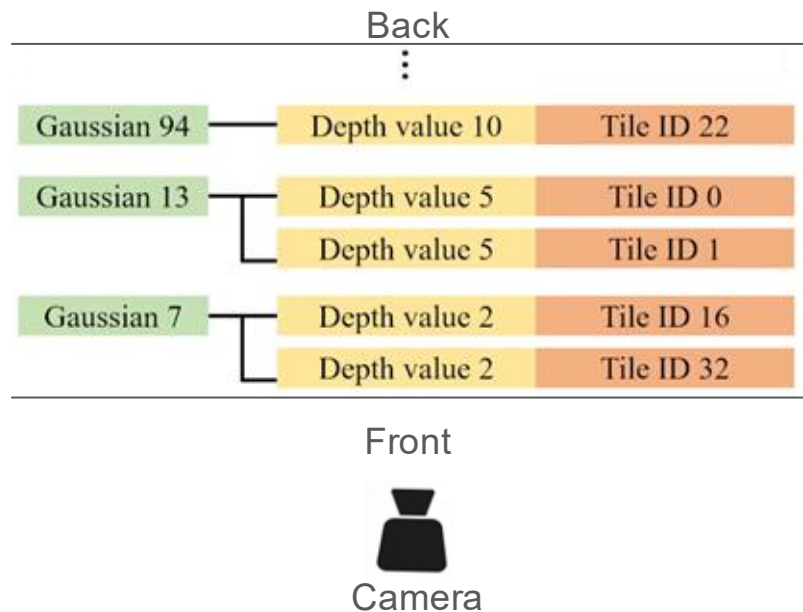
C, A : Gaussian colors and opacities

V : view configuration of current camera

```
function RASTERIZE( $w, h, M, S, C, A, V$ )  
  CullGaussian( $p, V$ ) ▷ Frustum Culling  
   $M', S' \leftarrow$  ScreenspaceGaussians( $M, S, V$ ) ▷ Transform  
   $T \leftarrow$  CreateTiles( $w, h$ )  
   $L, K \leftarrow$  DuplicateWithKeys( $M', T$ ) ▷ Indices and Keys  
  SortByKey( $K, L$ ) ▷ Globally Sort  
   $R \leftarrow$  IdentifyTileRanges( $T, K$ )  
   $I \leftarrow 0$  ▷ Init Canvas  
  for all Tiles  $t$  in  $I$  do  
    for all Pixels  $i$  in  $t$  do  
       $r \leftarrow$  GetTileRange( $R, t$ )  
       $I[i] \leftarrow$  BlendInOrder( $i, L, r, K, M', S', C, A$ )  
    end for  
  end for  
  return  $I$   
end function
```

Sort by Keys:

- Radix sort Gaussians based on their keys



3D Gaussian Splatting

Fast Differentiable Rasterizer

Algorithm 2 GPU software rasterization of 3D Gaussians

w, h : width and height of the image to rasterize

M, S : Gaussian means and covariances in world space

C, A : Gaussian colors and opacities

V : view configuration of current camera

```
function RASTERIZE( $w, h, M, S, C, A, V$ )  
  CullGaussian( $p, V$ )                                ▶ Frustum Culling  
   $M', S' \leftarrow$  ScreenspaceGaussians( $M, S, V$ )      ▶ Transform  
   $T \leftarrow$  CreateTiles( $w, h$ )  
   $L, K \leftarrow$  DuplicateWithKeys( $M', T$ )             ▶ Indices and Keys  
  SortByKeys( $K, L$ )                                    ▶ Globally Sort  
   $R \leftarrow$  IdentifyTileRanges( $T, K$ )  
   $I \leftarrow 0$                                        ▶ Init Canvas  
  for all Tiles  $t$  in  $I$  do  
    for all Pixels  $i$  in  $t$  do  
       $r \leftarrow$  GetTileRange( $R, t$ )  
       $I[i] \leftarrow$  BlendInOrder( $i, L, r, K, M', S', C, A$ )  
    end for  
  end for  
  return  $I$   
end function
```

Identify tile ranges:

- Produce a list of Gaussians for each tile by identifying the first and last depth-sorted entry that splats to a given tile.

3D Gaussian Splatting

Fast Differentiable Rasterizer

Algorithm 2 GPU software rasterization of 3D Gaussians

w, h : width and height of the image to rasterize

M, S : Gaussian means and covariances in world space

C, A : Gaussian colors and opacities

V : view configuration of current camera

```
function RASTERIZE( $w, h, M, S, C, A, V$ )  
  CullGaussian( $p, V$ )                                ▶ Frustum Culling  
   $M', S' \leftarrow$  ScreenspaceGaussians( $M, S, V$ )      ▶ Transform  
   $T \leftarrow$  CreateTiles( $w, h$ )  
   $L, K \leftarrow$  DuplicateWithKeys( $M', T$ )             ▶ Indices and Keys  
  SortByKeys( $K, L$ )                                   ▶ Globally Sort  
   $R \leftarrow$  IdentifyTileRanges( $T, K$ )  
   $I \leftarrow 0$                                        ▶ Init Canvas  
  for all Tiles  $t$  in  $I$  do  
    for all Pixels  $i$  in  $t$  do  
       $r \leftarrow$  GetTileRange( $R, t$ )  
       $I[i] \leftarrow$  BlendInOrder( $i, L, r, K, M', S', C, A$ )  
    end for  
  end for  
  return  $I$   
end function
```

Alpha Blending

- For each pixel, accumulate the color and α values by traversing Gaussian list front to back

$$C \approx \sum_{i=1}^N T_i \alpha_i c_i$$

weights colors

- When target saturation of α is reached for a pixel, stop accumulating for that pixel thread.

3D Gaussian Splatting

Fast Differentiable Rasterizer

Algorithm 2 GPU software rasterization of 3D Gaussians

w, h : width and height of the image to rasterize

M, S : Gaussian means and covariances in world space

C, A : Gaussian colors and opacities

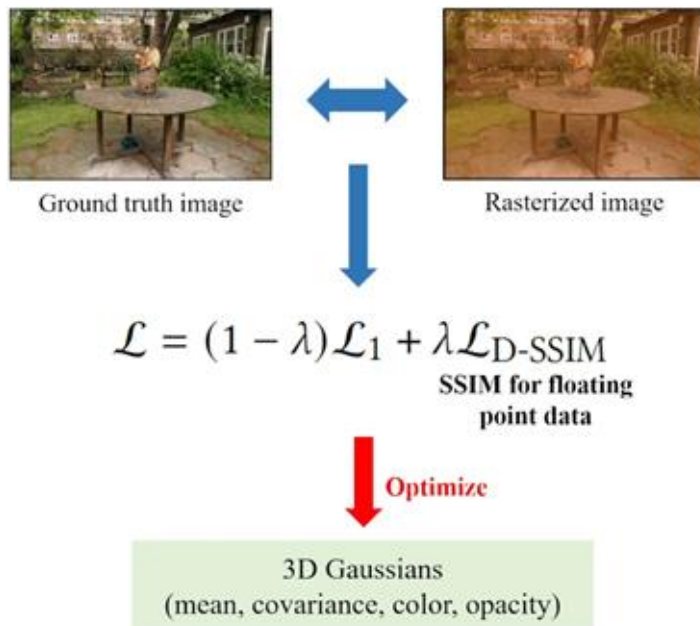
V : view configuration of current camera

```
function RASTERIZE( $w, h, M, S, C, A, V$ )  
  CullGaussian( $p, V$ )                                ▶ Frustum Culling  
   $M', S' \leftarrow$  ScreenspaceGaussians( $M, S, V$ )      ▶ Transform  
   $T \leftarrow$  CreateTiles( $w, h$ )  
   $L, K \leftarrow$  DuplicateWithKeys( $M', T$ )             ▶ Indices and Keys  
  SortByKey( $K, L$ )                                    ▶ Globally Sort  
   $R \leftarrow$  IdentifyTileRanges( $T, K$ )  
   $I \leftarrow 0$                                        ▶ Init Canvas  
  for all Tiles  $t$  in  $I$  do  
    for all Pixels  $i$  in  $t$  do  
       $r \leftarrow$  GetTileRange( $R, t$ )  
       $I[i] \leftarrow$  BlendInOrder( $i, L, r, K, M', S', C, A$ )  
    end for  
  end for  
  return  $I$   
end function
```

Return Image:

- Return rasterized image.

Compute loss:



3D Gaussian Splatting

Qualitative Results



Final RGB renders for novel views



RGB image and its' 3D Gaussians

3D Gaussian Splatting

Qualitative Results



Final RGB renders for novel views



Final RGB renders with reduced 2d Gaussian scales –
Shows compactness of Gaussians

3D Gaussian Splatting

Quantitative Results

Dataset Method Metric	Mip-NeRF360						Tanks&Temples					
	<i>SSIM</i> [↑]	<i>PSNR</i> [↑]	<i>LPIPS</i> [↓]	Train	FPS	Mem	<i>SSIM</i> [↑]	<i>PSNR</i> [↑]	<i>LPIPS</i> [↓]	Train	FPS	Mem
Plenoxels	0.626	23.08	0.463	25m49s	6.79	2.1GB	0.719	21.08	0.379	25m5s	13.0	2.3GB
INGP-Base	0.671	25.30	0.371	5m37s	11.7	13MB	0.723	21.72	0.330	5m26s	17.1	13MB
INGP-Big	0.699	25.59	0.331	7m30s	9.43	48MB	0.745	21.92	0.305	6m59s	14.4	48MB
M-NeRF360	0.792 [†]	27.69 [†]	0.237 [†]	48h	0.06	8.6MB	0.759	22.22	0.257	48h	0.14	8.6MB
Ours-7K	0.770	25.60	0.279	6m25s	160	523MB	0.767	21.20	0.280	6m55s	197	270MB
Ours-30K	0.815	27.21	0.214	41m33s	134	734MB	0.841	23.14	0.183	26m54s	154	411MB

Dataset Method Metric	Deep Blending					
	<i>SSIM</i> [↑]	<i>PSNR</i> [↑]	<i>LPIPS</i> [↓]	Train	FPS	Mem
Plenoxels	0.795	23.06	0.510	27m49s	11.2	2.7GB
INGP-Base	0.797	23.62	0.423	6m31s	3.26	13MB
INGP-Big	0.817	24.96	0.390	8m	2.79	48MB
M-NeRF360	0.901	29.40	0.245	48h	0.09	8.6MB
Ours-7K	0.875	27.78	0.317	4m35s	172	386MB
Ours-30K	0.903	29.41	0.243	36m2s	137	676MB

3D Gaussian Splatting

Limitations – Lack of Geometric regularization



Bernhard Kerbl, Georgios Kopanas, Thomas Leimkühler, George Drettakis. "3D Gaussian Splatting for Real-Time Radiance Field Rendering" ACM Trans. Graph. 42.4 (2023): 139-1

3D Gaussian Splatting

Limitations – Lack of Geometric regularization

How can we constrain geometry?

- Explicit scene representation with Gaussians. Hence each primitive has orientation.
- Each primitive trivially has depth.
- Framework assumes 2 or more views.
- Easiest way is to impose view dependent constraints on these geometric properties



